

Wazuh Deployment and Investigation Guide

Architecture, dashboard workflow, workstation setup, recovery and troubleshooting

Version: week1-prod-92

Publication date: 2026-08-14

Classification: Student Training Material

Authorised synthetic training use only.

Contents

[1. Wazuh SOC Level 1 Handbook — Module 1](#)

[2. Wazuh Dashboard Tutorial 01 — Orientation and First Investigation](#)

[3. NeoLabs SOC L1 Workstation Compatibility](#)

[4. NeoLabs SOC L1 Backup and Recovery](#)

[5. NeoLabs Wazuh Setup and Troubleshooting Guide](#)

[6. NeoLabs Student Wazuh Stack](#)

Wazuh SOC Level 1 Handbook — Module 1

Architecture, Data Flow and the NeoLabs Student Deployment

NeoLabs tested baseline: Wazuh 4.14.7

Module version: 1.0

Reconciled: 14 August 2026

Audience: Beginner-to-intermediate SOC Level 1 interns

Wazuh is not one program running in one window. It is a set of components that collect, analyse, store and display security data. A SOC analyst should be able to identify which stage produced—or failed to produce—the evidence they are looking for.

For the current programme/runtime summary, also read

`../../../../PROGRAMME_CURRENT_STATE.md`.

Learning objectives

By the end of this module, you should be able to:

- explain the roles of the Wazuh manager, Filebeat, indexer and dashboard;
 - explain how the NeoLabs VCC telemetry feed differs from a normal Wazuh endpoint agent;
 - describe how a VCC event becomes a searchable Wazuh alert;
 - distinguish source `event_time`, replay/ingestion metadata and Wazuh index time;
 - explain why an empty dashboard is not automatically evidence that no event occurred;
 - use the current NeoLabs one-click Windows startup and Doctor workflow;
 - explain the Night Watch and Telemetry Health views;
 - identify the student/operator trust boundary and pod-isolation controls;
 - perform a safe first-line data-flow check without exposing credentials.
-

1. What Wazuh is

Wazuh is an open-source security platform that can collect and analyse operating-system, application, network, cloud and security telemetry. In a typical deployment it combines endpoint agents/collectors, a Wazuh server or manager, Filebeat, a Wazuh indexer and a Wazuh dashboard.

For SOC work, think of the stack as a pipeline:

```
source event
→ collection
→ parsing/decoding
→ rule evaluation
→ alert creation
→ forwarding
→ indexing
→ search/dashboard
```

A failure at one stage can make later stages look empty even when earlier stages are healthy. That is why NeoLabs troubleshooting verifies the full chain rather than only checking that Docker containers are running.

2. Core Wazuh components

2.1 Wazuh agent

A Wazuh agent is software installed on a monitored endpoint. It can collect operating-system/application logs, Windows Event channels, file-integrity activity, system inventory and other endpoint security information.

NeoLabs VCC boundary

SOC interns do **not** install or administer Wazuh agents inside VCC application pods. The VCC remains operator-managed. The programme exports approved synthetic pod telemetry through a separate protected NeoLabs feed.

Therefore:

- absence of a pod-named Wazuh agent does not mean the VCC feed is absent;
- the student does not receive pod SSH/container access merely because Wazuh can see pod telemetry;
- student pod scope is controlled by the NeoLabs access service, not by a local Wazuh agent selector.

2.2 Wazuh manager

The Wazuh manager analyses received events. Important responsibilities include:

- reading configured log sources;
- decoding/parsing events;
- applying rules and correlations;
- writing alert records;
- coordinating supported Wazuh functions and API access.

The NeoLabs Compose service is:

```
wazuh.manager
```

The current manager configuration reads the shared NeoLabs VCC NDJSON stream at:

```
/var/ossec/logs/vcc/vcc-events.ndjson
```

with JSON log format.

2.3 Filebeat

Filebeat forwards Wazuh alerts from the manager toward the indexer.

This creates an important troubleshooting distinction:

```
manager parsed/matched the event  
but  
Filebeat/indexer did not make it searchable
```

That situation is different from a collector failure or a decoder/rule failure.

2.4 Wazuh indexer

The Wazuh indexer is the search/storage layer based on OpenSearch technology. It stores documents, mappings and searchable indices.

The NeoLabs Compose service is:

```
wazuh.indexer
```

Week 1 investigation primarily uses:

```
wazuh-alerts-*
```

Normal NeoLabs VCC baseline events are deliberately eligible to become searchable alerts so Operation Night Watch can show normal activity, not only high-severity detections.

2.5 Wazuh dashboard

The Wazuh dashboard provides the analyst interface for search, Threat Hunting/Discover, saved objects, dashboards and Wazuh modules.

The NeoLabs service is:

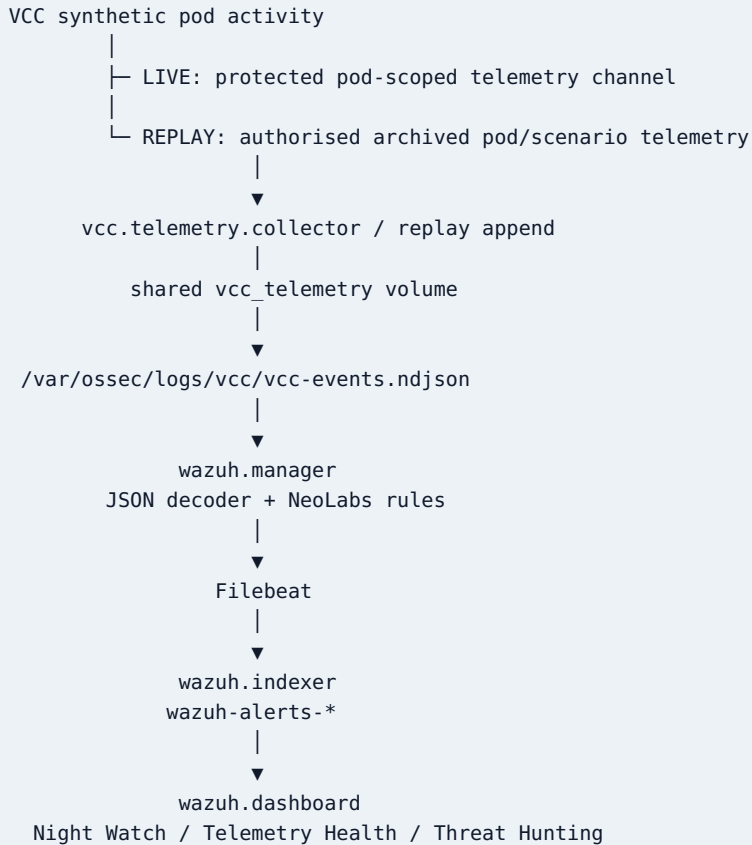
```
wazuh.dashboard
```

The normal student profile binds the dashboard to loopback:

```
https://127.0.0.1:8443
```

The dashboard, Wazuh API and indexer must not be exposed publicly merely to simplify student access.

3. The current NeoLabs student topology



Persistence

Docker named volumes retain manager/indexer/dashboard state across ordinary stops. A normal `git pull` or stack restart does not require students to delete Wazuh volumes or regenerate credentials.

A destructive reset is a separate local-only operation and is **not** normal troubleshooting.

4. How VCC telemetry reaches the student

The programme separates **telemetry access** from **infrastructure access**.

A SOC intern receives:

- the SOC toolkit;
- a private NeoLabs Access Code;
- the server-assigned pod/track/scenario context;
- a local Wazuh workstation;
- only the synthetic telemetry/evidence authorised for that assignment.

The intern does **not** receive:

- EC2 or pod shell access;
- database/container administration;
- broad AWS credentials;
- another pod's telemetry;
- mentor ground truth;
- a client-side control that authorises a different pod.

Server-authoritative scope

The student may type a pod number during login as confirmation, but the server-side assignment is authoritative. A local edit cannot turn a student into another pod/track.

Both live and replay paths validate the assigned scope. Replay additionally validates that records are synthetic, belong to the assigned pod and contain required event fields before they are appended to the local Wazuh telemetry stream.

5. LIVE versus REPLAY

The VCC uses a cost-aware hybrid runtime. The main VCC EC2 does not have to remain on for every hour of an investigation.

The same student access flow can represent different authorised SOC states:

- **LIVE** — protected live pod telemetry during approved interactive windows;
- **REPLAY** — archived telemetry for the assigned pod/scenario;
- **CLOUD_LIVE / ENDPOINT_LIVE** — scenario-specific cloud/endpoint evidence surfaces where applicable;
- **OFFLINE/no surface** — the toolkit must not pretend data is available.

Event time during replay

Replay preserves the original source:


```
event_time
```

and adds replay metadata separately.

For an incident timeline, order events by original event time unless the assignment specifically asks you to analyse collection/replay delay.

6. How Wazuh analyses a NeoLabs event

A simplified event path is:

```
NDJSON event
→ JSON decoding
→ NeoLabs base rule
→ child/correlation rules where applicable
→ alert record
→ Filebeat
→ Wazuh indexer
→ dashboard search
```

A synthetic VCC event may contain fields such as:

```
{
  "schema_version": "1.0",
  "event_id": "evt-auth-0009",
  "event_time": "2026-08-14T09:18:07Z",
  "pod_id": "pod-03",
  "event_type": "authentication",
  "action": "login",
  "outcome": "success",
  "user": "student.synthetic",
  "source_ip": "203.0.113.44",
  "session_id": "sess-902",
  "correlation_id": "corr-902",
  "synthetic": true
}
```

Wazuh may expose NeoLabs dynamic fields under `data.*`. In the current mapping, the NeoLabs user identity is commonly visible as `data.dstuser` in the saved Week 1 view.

7. NeoLabs custom rules

The current NeoLabs VCC rule set includes training rules for:

- normal VCC baseline events;
- authentication failures;
- repeated authentication failures;
- successful authentication;
- success after earlier failures;
- sensitive account changes;
- protected authorisation denials;
- telemetry collection/parser/visibility problems.

The base VCC event rule is intentionally searchable at a non-zero alert level so normal Week 1 activity can appear in `wazuh-alerts-*`.

Telemetry-health problems are represented around rule:

```
100150
```

A Wazuh rule match means the configured condition was observed. It does **not** automatically prove malicious intent or impact.

8. Current Windows setup and startup

The normal student workflow has changed from the older manual enrolment sequence.

First/current startup

From the latest toolkit checkout:

1. Start Docker Desktop with WSL2 integration.
2. Double-click:

```
START-NEOLABS-SOC.cmd
```

The launcher automatically:

1. verifies WSL2/toolkit prerequisites;
2. performs first-time Wazuh preparation only when needed;
3. preserves an existing `.env` and Wazuh data;

4. reuses a valid NeoLabs session or asks for assigned pod + private Access Code;
5. connects to the current authorised LIVE/REPLAY surface;
6. starts/waits for manager, indexer, dashboard and telemetry collector;
7. reloads the current NeoLabs rule file in the manager;
8. proves an assigned-pod VCC event is searchable in `wazuh-alerts-*`;
9. reports latest-event freshness;
10. applies/checks local alert-index retention/disk safety;
11. provisions Night Watch/Telemetry Health saved objects where supported;
12. copies the local Wazuh `admin` password to the Windows clipboard without printing it;
and
13. opens the local dashboard.

Do not begin cohort analysis until the launcher displays:

```
SOC WORKSTATION READY
```

No global CLI/manual gateway requirement

For the normal Windows programme path, students do **not** need to:

- run `python -m pip install -e .`;
- add Python Scripts to Windows PATH;
- copy a private/manual gateway URL;
- manually enrol with a token file;
- manually select another telemetry target.

Advanced/manual scripts still exist for troubleshooting and non-Windows operation, but they are not the primary student onboarding sequence.

9. Wazuh dashboard login

After READY, the normal dashboard is:

```
https://127.0.0.1:8443
```

Use:

```
Username: admin  
Password: the locally generated WAZUH_INDEXER_PASSWORD
```

On Windows, the launcher copies the password to the clipboard without displaying it. Press `Ctrl+V` at the login page, then replace the clipboard contents with non-sensitive text after login.

The following are **not** the human dashboard password:

- `WAZUH_API_PASSWORD` — internal Wazuh API/service communication;
- `WAZUH_DASHBOARD_PASSWORD` — internal dashboard service account.

Never paste the local Wazuh password into an assignment, screenshot, Slack post or Git commit.

10. Night Watch and Telemetry Health views

NeoLabs — Operation Night Watch

The toolkit attempts to provision a Week 1 pod-scoped saved view/dashboard containing important investigation fields such as:

- `data.pod_id`;
- `data.event_type`;
- synthetic user identity;
- `data.source_ip`;
- `data.outcome`;
- `data.correlation_id`;
- Wazuh `rule.id`, `rule.level`, `rule.description`;
- original `data.event_time`.

Use it to establish normal authentication/application/API behaviour and build reusable Week 1 searches.

NeoLabs — Telemetry Health

This view focuses on telemetry-quality problems, especially rule `100150`, so students can distinguish “no suspicious event” from “the collection/search pipeline is unhealthy.”

Saved-object provisioning is fail-soft. If an import is unavailable on a local dashboard instance, Threat Hunting/Discover remains the supported fallback.

11. Freshness and local retention

The current toolkit reports the newest assigned-pod event indexed in Wazuh.

Default freshness warning:

```
90 minutes
```

This is a health signal—not proof that an attack should happen every 90 minutes.

Current local alert-index defaults:

```
wazuh-alerts-* retention: 30 days
filesystem warning:      85%
filesystem critical:     92%
```

This retention applies only to the student's local Wazuh alert indices. It does **not** delete VCC server-side telemetry archives/evidence and does not force-overwrite unrelated existing index-management policies.

12. NeoLabs Doctor

On Windows, double-click:

```
CHECK-NEOLABS-SOC.cmd
```

or run:

```
.\neolabs.cmd doctor
```

Doctor checks the pipeline in order:

1. NeoLabs authentication
2. current LIVE/REPLAY telemetry surface
3. raw VCC event file
4. Wazuh rule engine
5. Filebeat
6. Wazuh indexer
7. Wazuh dashboard

It also reports telemetry freshness and local index/disk state.

This is the preferred first troubleshooting tool because it prevents the common mistake of changing Wazuh rules when the real failure is earlier in the pipeline.

13. Why an empty dashboard may be misleading

Before writing “no events were found,” confirm:

1. Doctor/indexer health passes;
2. the latest VCC event is reasonably fresh for the current assignment/window;
3. the filter uses the assigned `pod_id`;
4. the correct `wazuh-alerts-*` data view is selected;
5. the time range includes the original event times;
6. Telemetry Health does not show a collection/parser/visibility problem.

A zero-result query becomes useful evidence only after these assumptions are checked.

14. Alerts versus archives

`wazuh-alerts-*` contains searchable alert documents. The current NeoLabs Week 1 base rule ensures normal approved VCC baseline events can be represented there.

Wazuh archive indexing, when enabled, can provide additional events that did not become alerts. Archive indexing may not be available on every student workstation because it increases storage/resource/privacy requirements.

Do not assume `wazuh-archives-*` exists. If the assignment needs evidence that is not available in alerts, record the visibility limitation and use only mentor-approved evidence sources.

15. Network exposure and security boundaries

Common Wazuh ports can include agent communication/enrolment, server API, indexer and dashboard ports. In the NeoLabs student workstation:

- the dashboard is loopback-only by default;
- the Wazuh API/indexer are not intentionally published for student network access;
- internal components communicate on the Compose network;
- VCC telemetry scope is server-managed;
- students must not expose the local stack publicly for convenience.

The fact that Wazuh is a security tool does not make an insecure Wazuh deployment acceptable.

16. Safe manual checks

Advanced/manual checks remain available from the toolkit root, for example:

```
bash wazuh-stack/scripts/compatibility-check.sh
bash wazuh-stack/scripts/health-check.sh
bash wazuh-stack/scripts/verify-telemetry-pipeline.sh --wait 180
bash wazuh-stack/scripts/telemetry-freshness.sh
bash wazuh-stack/scripts/doctor.sh
```

Use `wazuh-logtest` through the supported verifier/Doctor workflow when checking decoding/rules. Do not broadly raise rule levels, disable TLS, publish indexer/API ports or delete volumes simply to make a dashboard show data.

17. Evidence and confidentiality

Useful Week 1 evidence includes:

- Wazuh event/rule IDs;
- assigned pod and explicit time range;
- relevant source event fields;
- saved/reproducible filters;
- original event-time timeline;

- telemetry-health/freshness notes;
- redacted screenshots.

Never submit:

- NeoLabs Access Code;
- Wazuh local password;
- session token;
- signed private URL;
- private key/certificate contents;
- AWS credentials;
- another pod's data;
- real customer/production information.

18. Week 1 practical checklist

For Operation Night Watch:

1. Start with `START-NEOLABS-SOC.cmd`.
2. Wait for `SOC WORKSTATION READY`.
3. Log in to the local Wazuh dashboard as `admin`.
4. Open NeoLabs — Operation Night Watch or Threat Hunting.
5. Confirm your assigned pod filter.
6. Record latest-event freshness and inspect Telemetry Health.
7. Find normal authentication and application/API activity.
8. Pivot using identity/source/session/correlation fields.
9. Save/reuse at least three baseline searches.
10. Build a short original-event-time timeline.
11. Record one visibility limitation.
12. Submit only redacted evidence through the central assignments repository.

19. Completion standard

You understand the NeoLabs Wazuh architecture when you can answer all of these without guessing:

- Where did this event originate?
- How did it reach the local telemetry file?
- Did Wazuh decode and match a rule?
- Did Filebeat/indexer make it searchable?

- Am I using the correct assigned pod and time range?
- Is the newest telemetry fresh/healthy?
- Is this rule an observation or proof of compromise?
- What can the current telemetry prove, and what can it not prove?

That pipeline awareness is one of the most important habits of a Level 1 SOC analyst.

Source: docs/dashboard-tutorials/01-orientation-and-alert-investigation.md

Wazuh Dashboard Tutorial 01 — Orientation and First Investigation

NeoLabs tested baseline: Wazuh 4.14.7

Reconciled: 2026-08-14

Audience: SOC Level 1 interns

Dashboard labels can vary by release. Current programme/startup behaviour is documented in `../../../../PROGRAMME_CURRENT_STATE.md`.

Before opening the dashboard

On Windows, normally start with:

```
START-NEOLABS-SOC.cmd
```

Wait for `SOC WORKSTATION READY`. That means the toolkit has already verified service health **and** that assigned-pod VCC telemetry is searchable in the local Wazuh indexer.

If anything looks wrong:

```
CHECK-NEOLABS-SOC.cmd
```

or:

```
.\neolabs.cmd doctor
```

Normal dashboard URL:

```
https://127.0.0.1:8443
```

Login username is `admin`; the one-click Windows launcher copies the locally generated password to the clipboard without printing it.

Current NeoLabs investigation views

NeoLabs — Operation Night Watch

Use this preconfigured pod-scoped saved view/dashboard for Week 1 when provisioning succeeds. It focuses on:

- assigned `pod_id`;
- `event_type`;
- NeoLabs user identity (`data.dstuser` in the current Wazuh mapping);
- `source_ip`;
- `outcome`;
- `correlation_id`;
- Wazuh rule ID/level/description;
- original source `event_time`.

NeoLabs — Telemetry Health

Use this saved view for collector/parser/visibility problems, especially NeoLabs rule `100150`.

Saved-object provisioning is fail-soft. If these views are unavailable, use **Threat Hunting/Discover** against `wazuh-alerts-*`; do not reset Wazuh just because a saved view did not import.

Alerts versus raw/archive data

The current NeoLabs Week 1 workflow begins with `wazuh-alerts-*`. Normal VCC baseline events are deliberately eligible for searchable alerts so interns can see normal Night Watch activity, not only high-severity detections.

`wazuh-archives-*` may not be enabled on every student workstation. If raw/archive data is unavailable, state that visibility limitation rather than assuming the event never existed.

Time is evidence

The global time picker is one of the most common causes of false “no data” conclusions.

For assignment evidence:

1. locate the relevant event(s);
2. choose an absolute UTC range around their **original event time**;
3. record that range in the query journal;
4. distinguish source `event_time` from replay/ingest/indexed timestamp.

Replay preserves original `event_time` and stores replay metadata separately.

First Week 1 investigation flow

1. Open **NeoLabs — Operation Night Watch** or Threat Hunting.
2. Confirm the assigned `pod_id`. If another pod appears, stop and contact a mentor.
3. Check the latest event freshness reported by the launcher/Doctor.
4. Inspect any Telemetry Health warning before interpreting missing data.
5. Set an absolute time range around the Week 1 events.
6. Expand a record and capture `event_id`, `original_event_time`, `event_type`, `identity`, `source`, `outcome`, `session/correlation IDs` and Wazuh rule metadata where available.
7. Find normal successful authentication and an ordinary failed authentication if present.
8. Pivot by identity → source IP → session/correlation ID to related application/API activity.
9. Identify storage/other approved telemetry visible to the pod.
10. Save/reuse at least three baseline filters/searches.
11. Build a short original-event-time timeline.
12. Record one visibility gap.

Week 1 is baseline building. “Unusual” does not automatically mean “malicious.”

Useful NeoLabs rule families

Current training rules include normal baseline events plus authentication failure/repeated failure, authentication success/success-after-failures, sensitive account change, authorisation denial and telemetry-health alerts.

Rule IDs/severity are pivots. Always inspect underlying event fields/context before disposition.

Filters and pivots

Useful pivots include assigned pod, identity, source IP, event type, outcome, session/correlation ID and rule ID. Before trusting a result count, inspect every active filter and the current time range.

A practical sequence:

```
pod
→ identity
→ source_ip
→ session_id / correlation_id
→ related application/auth/account activity
→ telemetry health for the same window
```

Do not reproduce denied/cross-pod requests merely to create evidence.

WQL/search-bar caution

Wazuh Query Language, OpenSearch/Discover query syntax and UI filter pills are not interchangeable everywhere. Prefer UI field filters when following this Week 1 tutorial. If you use a query language, record exactly which view/search bar accepted it so another analyst can reproduce the result.

Zero results are not automatically evidence of absence

Before writing “no events found”:

- confirm Doctor/indexer PASS;
- confirm latest VCC event freshness;
- confirm the correct data view/index pattern;
- confirm the assigned pod filter;
- confirm absolute time range/time zone;
- inspect Telemetry Health / rule 100150 ;
- state missing archive/source coverage if relevant.

Evidence capture

A useful screenshot should show enough context to prove the claim: view/search name, time range, active filters and relevant event/rule fields. Redact local Wazuh password, NeoLabs Access Code/session, signed URLs, private keys/certificates and unrelated personal data.

For every material conclusion, retain the event/evidence ID or reproducible search in the query journal.

Dashboard/agent distinction

The NeoLabs VCC pod feed is consumed as a protected telemetry stream/local file path; it is not permission to install/manage an agent inside a VCC pod. Absence of a pod-named Wazuh agent does not mean the VCC feed is absent.

When to use Doctor instead of changing settings

If you suspect ingestion/indexing failure, run `CHECK-NEOLABS-SOC.cmd / .\neolabs.cmd doctor` before editing dashboard/indexer/manager settings. The Doctor is designed to identify whether the break is at authentication, replay/live access, raw event, rule engine, Filebeat, indexer or dashboard.

Do not weaken TLS, expose indexer/API ports, disable rules broadly or delete local volumes simply to make a dashboard show data.

Completion check

You can complete this tutorial when you can explain the assigned pod, current data source/time range, one normal auth sequence, one application/API pivot, the meaning of the relevant Wazuh rule, one saved/reproducible filter and one visibility limitation—without confusing replay/index time with source event time.

Source: docs/setup/WORKSTATION_COMPATIBILITY.md

NeoLabs SOC L1 Workstation Compatibility

For current programme/startup behaviour see [../../../../PROGRAMME_CURRENT_STATE.md](#).

Supported baseline

Run before/when troubleshooting setup:

```
bash wazuh-stack/scripts/compatibility-check.sh
```

Requirement	Hard floor	Preferred minimum	Recommended
CPU architecture	64-bit x86_64 or arm64	64-bit x86_64	x86_64
CPU capacity	4 logical cores	4	6+
Memory visible to Linux/WSL2	7 GiB	8 GiB	12–16 GiB
Free disk	25 GiB	25 GiB	50 GiB
Container runtime	Docker Engine/Desktop	Docker Engine/Desktop	current supported release
Compose	v2	v2	current supported v2
<code>vm.max_map_count</code>	262144	262144	≥ 262144

Seven GiB is accepted as the constrained Week 1 floor. Below 7 GiB is unsupported because manager/indexer/dashboard may become unstable. Python 3, OpenSSL and curl are also required by toolkit validation/access tooling.

Windows 11 + WSL2 — recommended cohort path

WSL2 is required for the Linux Wazuh stack, but Ubuntu is **not** mandatory. Kali, Ubuntu, Debian or another current WSL2 distribution is acceptable when Bash, Docker access, Python 3, OpenSSL and curl are present.

Enable Docker Desktop WSL integration for the distro being used. The current Windows entry point is:

```
START-NEOLABS-SOC.cmd
```

The launcher/setup checks the workstation, prepares the stack only if needed and runs the Linux tooling through WSL2. Do not run the Wazuh stack directly from Git Bash/MSYS/Cygwin.

An existing toolkit checkout on the Windows filesystem can work when WSL2/Docker can access it; for best Linux container performance/permissions, a WSL/Linux filesystem checkout is preferred. Do not move a working cohort checkout mid-assignment merely to satisfy a preference if the compatibility/startup checks already pass.

CRLF issue

The repository forces LF endings for shell scripts. If an old checkout reports `pipefail\r/` invalid option errors, repair line endings or re-checkout safely. Do not run `git reset --hard` when uncommitted student evidence/work must be preserved.

Linux

Current Linux distributions are supported when Docker/Compose prerequisites and kernel settings pass. If required:

```
sudo sysctl -w vm.max_map_count=262144
```

Make persistence changes using the OS-approved sysctl mechanism.

macOS

Intel Docker Desktop can be used when memory/disk requirements are satisfied. Apple Silicon is a review-warning path because each pinned Wazuh image must support the architecture; use a supported x86_64 Linux/WSL2 workstation when the pinned image set does not.

Unsupported configurations

- 32-bit operating systems;
- legacy Docker Toolbox/Compose v1;
- browser-only/mobile environments;
- less than 7 GiB Linux/WSL2-visible memory;
- systems where the student cannot use local Docker;
- production/company shared infrastructure;
- public cloud hosts exposed as a student Wazuh server;
- Git Bash/MSYS/Cygwin as the Wazuh runtime.

Current pre-start checklist

Windows normal path

1. Pull the latest toolkit.
2. Start Docker Desktop/WSL2 integration.
3. Double-click `START-NEOLABS-SOC.cmd`.
4. Resolve any compatibility/setup failure it reports.
5. Do not begin Week 1 until `SOC WORKSTATION READY` confirms assigned-pod VCC telemetry is searchable.

Manual/advanced path

1. Run compatibility check.
2. Prepare/reuse `.env` and generated Wazuh configuration.
3. Run preflight/start/health checks.
4. Use the NeoLabs toolkit access client for current LIVE/REPLAY assignment.
5. Run the telemetry pipeline verifier/Doctor before treating missing Wazuh data as an investigation result.

A warning may be accepted only when its risk is understood/documented. A failure must be corrected before the learner relies on the workstation for assignment evidence.

Source: docs/setup/BACKUP_AND_RECOVERY.md

NeoLabs SOC L1 Backup and Recovery

Purpose

The student Wazuh environment contains locally generated investigation data, dashboards, indexer data and manager state. A workstation failure should not require a learner to rebuild every investigation from the beginning.

The toolkit therefore provides a guarded Docker-volume backup and restore process. It is designed for local training data, not for production disaster recovery.

Security boundary

A toolkit backup deliberately excludes:

- `.env` ;
- VCC bootstrap tokens;
- client private keys;
- client certificates and CA trust files;
- enrolment responses;
- private VCC endpoints;
- production data or credentials.

After restoring, the learner must complete a fresh operator-approved enrolment. This prevents copied backups from becoming reusable VCC access packages.

Create a backup

From the repository root:

```
bash wazuh-stack/scripts/backup.sh
```

The script performs the following actions:

1. confirms that Docker, Compose and the local `.env` file are available;
2. discovers the named volumes declared by the Wazuh Compose file;
3. stops the running stack to obtain a consistent point-in-time snapshot;
4. creates one compressed archive per existing volume;
5. calculates SHA-256 checksums;
6. writes a manifest and recovery notice;
7. restarts the stack unless `--no-restart` was supplied.

Use a specific destination when required:

```
bash wazuh-stack/scripts/backup.sh --output /secure/local/path/neolabs-backup
```

Keep the backup on encrypted, personally controlled storage. Do not upload it to a public repository or share it in Slack or WhatsApp.

Verify a backup

Always verify a backup before relying on it:

```
bash wazuh-stack/scripts/verify-backup.sh /path/to/backup
```

Verification checks the manifest version, checksum file, archive readability, presence of at least one volume and absence of common credential filenames.

Restore a backup

A restore replaces the contents of the matching Wazuh volumes. Stop the stack and confirm the backup path before continuing:

```
bash wazuh-stack/scripts/stop.sh  
bash wazuh-stack/scripts/restore.sh /path/to/backup --force
```

The restore script:

- verifies checksums before modifying volumes;
- accepts only volume names declared in the current Compose file;
- refuses to run while stack services are active;
- recreates and repopulates the declared volumes;
- removes stale local VCC collector scope and cursor state;
- does not restore enrolment credentials.

After restoration:

```
bash wazuh-stack/scripts/compatibility-check.sh  
bash wazuh-stack/scripts/start.sh  
bash wazuh-stack/scripts/health-check.sh
```

Then request a new enrolment token from the programme operator and complete the normal enrolment workflow.

Rehearsal

The repository CI performs a synthetic Docker-volume recovery rehearsal using:

```
bash wazuh-stack/tests/backup-restore-rehearsal.sh
```

The rehearsal creates a temporary volume, writes a synthetic marker, archives it, verifies its checksum, deletes and recreates the volume, restores the archive and confirms that the marker survived. It never reads the learner's Wazuh data.

Recovery limitations

A successful archive verification does not prove that every future Wazuh release can read data created by an older release. Restore into the same pinned toolkit version first. Complete a documented Wazuh upgrade only after the restored stack is healthy.

Backups are not a substitute for GitHub submissions. Investigation reports, evidence logs and query journals should still be committed to the approved assignment repository after removing credentials, private URLs and sensitive evidence.

Source: troubleshooting/WAZUH_SETUP_AND_TROUBLESHOOTING_GUIDE.md

NeoLabs Wazuh Setup and Troubleshooting Guide

Tested baseline: Wazuh 4.14.7

Reconciled: 2026-08-14

Scope: learner-owned workstation + authorised synthetic VCC telemetry only

Current Windows first response is `CHECK-NEOLABS-SOC.cmd` / `.\neolabs.cmd doctor`.
Troubleshoot from the VCC source toward the dashboard and change one layer at a time.

1. Current golden path

```
NeoLabs authentication / server assignment
→ current LIVE/REPLAY surface
→ raw assigned-pod VCC event file
→ Wazuh manager localfile input
→ NeoLabs Wazuh rule engine
→ Filebeat
→ Wazuh indexer (wazuh-alerts-*)
→ dashboard / saved views / time filters
```

The one-click launcher will not display `SOC WORKSTATION READY` unless the assigned-pod telemetry is actually searchable in the indexer.

2. Run Doctor first

Windows

Double-click:

```
CHECK-NEOLABS-SOC.cmd
```

or:

```
.\neolabs.cmd doctor
```

Doctor prints PASS/WARN/FAIL by stage and reports the latest indexed VCC event freshness, local index size/disk state and dashboard reachability.

Manual Linux/WSL inspection

From the toolkit root:

```
bash wazuh-stack/scripts/health-check.sh
bash wazuh-stack/scripts/verify-telemetry-pipeline.sh --wait 180
bash wazuh-stack/scripts/telemetry-freshness.sh
bash wazuh-stack/scripts/doctor.sh
```

Record useful non-secret output in your query journal. Never post complete `.env`, Docker environment dumps, Access Codes, passwords, private keys/certificates or private signed URLs.

3. Docker/WSL problems

Docker command/daemon unavailable

Check:

```
docker --version
docker compose version
docker info
```

On Windows, start Docker Desktop and confirm WSL integration for the distro in use. Do not expose an unauthenticated Docker TCP socket.

Git Bash/MSYS/Cygwin

Do not run the Wazuh stack directly there. Use `START-NEOLABS-SOC.cmd`, which hands Linux work into WSL2.

Memory/disk

Current hard memory floor is 7 GiB visible to Linux/WSL2; 8 GiB preferred; 12–16 GiB recommended. Free disk hard floor is 25 GiB; 50 GiB recommended.

Check:

```
docker stats --no-stream
docker system df
df -h
```

The toolkit also warns at 85% local indexer filesystem use and treats 92% as critical. Do not delete Wazuh volumes just to silence a disk warning without preserving required assignment evidence/backup.

vm.max_map_count

Check:

```
cat /proc/sys/vm/max_map_count
```

Expected at least `262144` where the platform exposes this setting.

4. First-run/local configuration problems

The Windows launcher calls the supported setup path when `wazuh-stack/.env` is absent and preserves an existing `.env`. Do not regenerate local secrets simply because the repo was updated.

Manual advanced preparation uses the scripts under `wazuh-stack/scripts/`; follow `wazuh-stack/README.md` rather than an old copied command sequence.

The human Wazuh login is:

```
Username: admin
Password source: local WAZUH_INDEXER_PASSWORD
```

The Windows launcher copies this password to the clipboard without printing it. `WAZUH_API_PASSWORD` and `WAZUH_DASHBOARD_PASSWORD` are internal service credentials and are not the human login.

5. NeoLabs authentication/runtime failures

Check:

```
.\neolabs.cmd status
.\neolabs.cmd pod info
```

If the saved session is expired/rejected, run `.\neolabs.cmd login` with the assigned pod + private Access Code. The pod supplied at login is only a confirmation value; server assignment remains authoritative.

Do not edit runtime files to reach another pod and do not manually replace the normal gateway URL using an old onboarding message.

6. Replay/live telemetry not arriving

Run Doctor. Then verify:

- server state is a valid SOC surface (`LIVE` , `REPLAY` , applicable cloud/endpoint state);

- raw telemetry contains synthetic records for the assigned pod;
- replay pack validation did not reject pod/scenario/event fields;
- in LIVE mode, the protected collector/enrolment state is current;
- internet/TLS connectivity is available.

Replay preserves original `event_time`. Do not treat replay time as the incident timeline.

7. Raw events exist but Wazuh has no alerts

The manager must monitor `/var/ossec/logs/vcc/vcc-events.ndjson` as JSON and load `config/rules/neolabs_vcc_rules.xml`.

The normal startup path restarts the manager after asserting the stack so the current NeoLabs rules are loaded after `git pull`.

Use `wazuh-logtest` through the verifier/Doctor before editing decoders/rules. The NeoLabs base VCC rule is intentionally searchable so normal Week 1 baseline events can reach `wazuh-alerts-*`.

Do not change rule levels broadly merely to make a dashboard look busy.

8. Filebeat/indexer problems

Symptoms include raw events/rule matches but no `wazuh-alerts-*` documents, dashboard index errors or indexer health failures.

Check Doctor and service logs. Verify the indexer is healthy before changing mappings/policies. Local retention is deliberately scoped to `wazuh-alerts-*`; unrelated ISM policies are not force-overridden.

If the disk is near watermarks, preserve/submit required evidence first, then follow mentor-approved local cleanup/retention guidance rather than deleting volumes blindly.

9. Dashboard opens but data looks empty

Before concluding “no events”:

1. confirm Doctor/indexer PASS;
2. check `Last VCC event indexed` freshness;
3. confirm the assigned `pod_id` filter;
4. use `wazuh-alerts-*` / Threat Hunting or the **NeoLabs — Operation Night Watch** view;
5. set an appropriate/absolute UTC time range around the source event times;
6. inspect the **NeoLabs — Telemetry Health** view/rule `100150` for collection/parser/visibility issues.

A zero-result query is evidence only after source health and time/filter assumptions are verified.

10. Saved Night Watch/Telemetry Health views missing

Saved-object provisioning is fail-soft. The telemetry/indexer can be healthy even if the dashboard API rejects an import on a particular local version/state.

Run Doctor and continue through Threat Hunting if needed. Do not reset the Wazuh data/indexer merely because a saved view is absent.

11. Safe bounded repair

The one-click launcher/repair tooling may reassert local services and, in replay mode, refetch authorised replay state once. It does **not** reset the VCC server, switch pods, delete Wazuh volumes or fetch another student's data.

If one bounded repair still cannot prove assigned-pod telemetry is searchable, stop relying on that workstation for assignment conclusions and send the Doctor output (redacted) to the mentor.

12. Dashboard certificate/browser warning

The local dashboard uses local TLS. Verify you are opening the expected loopback URL (`https://127.0.0.1:8443` by default) from your own workstation. Never work around a certificate warning by exposing the dashboard on a public interface.

13. Backups/resets

Use the documented backup/verify/restore scripts before destructive local changes.

`reset.sh --confirm-destroy-local-data` destroys local Wazuh state; it is not normal troubleshooting. Removing local VCC credential files does not by itself revoke server-side assignment/credentials.

14. Evidence to send a mentor

Useful redacted evidence:

- Doctor PASS/WARN/FAIL lines;
- current assigned pod/scenario/runtime state without Access Code/token;
- latest event freshness;
- `docker compose ps` status;
- small relevant service-log excerpts;
- event/rule IDs or a screenshot showing the failing dashboard stage.

Never send `.env`, copied admin password, Access Code, session token, private key/certificate contents or another pod's data.

15. Stop conditions

Stop and escalate on cross-pod events/access, real personal/production data, exposed credentials/private keys, unexpected infrastructure access, external-target traffic or service instability. Do not weaken isolation/security controls to make a troubleshooting check pass.

Source: wazuh-stack/README.md

NeoLabs Student Wazuh Stack

Purpose

This directory provides the local Wazuh manager, indexer and dashboard used by authorised NeoLabs SOC Level 1 interns. The stack is pinned to Wazuh 4.14.7 / Docker tag v4.14.7 and receives only synthetic telemetry authorised for the learner's server-assigned VCC pod.

For current programme/startup behaviour read [../PROGRAMME_CURRENT_STATE.md](#). The old release-candidate/manual-enrolment workflow is no longer the normal student path.

Recommended Windows path

Students normally do **not** enter this directory and manually perform every setup/enrolment step. From the toolkit root:

```
START-NEOLABS-SOC.cmd
```

The launcher prepares/reuses the stack, authenticates to NeoLabs, connects the assigned LIVE/REPLAY surface, waits for service health, reloads current NeoLabs rules, proves assigned-pod telemetry is searchable in `wazuh-alerts-*`, reports freshness/retention/disk state, provisions Night Watch/Telemetry Health saved objects where supported, copies the local `admin` password to the Windows clipboard without printing it and opens the dashboard.

If anything is wrong:

```
CHECK-NEOLABS-SOC.cmd
```

or:

```
.\neolabs.cmd doctor
```

Current readiness meaning

`SOC WORKSTATION READY` means more than containers are running. A real synthetic event for the current server-assigned pod must be present in the local telemetry path and searchable through the Wazuh indexer. If that proof fails, the launcher makes at most one bounded local repair attempt and does not falsely report READY.

Dashboard login

Normal local URL:

```
https://127.0.0.1:8443
```

Username is `admin`. The human dashboard password is the locally generated `WAZUH_INDEXER_PASSWORD` in the private `.env`; the Windows launcher copies it to the clipboard without printing it. The internal `wazuh-wui` and `kibanaserver` service passwords are separate and are not student login credentials.

Current telemetry path

```
VCC live/archive
→ NeoLabs authorised LIVE/REPLAY access
→ local vcc-events.ndjson
→ Wazuh manager JSON localfile input
→ NeoLabs VCC rules
→ Filebeat
→ Wazuh indexer wazuh-alerts-*
→ dashboard / Threat Hunting / saved views
```

Replay validates `synthetic=true`, assigned `pod_id`, required event fields and scenario scope before append. Original `event_time` is preserved; replay metadata is separate.

Current NeoLabs rules

`config/rules/neolabs_vcc_rules.xml` includes training rules for:

- normal VCC baseline events (searchable in `wazuh-alerts-*`);
- authentication failures;
- repeated failures by account/source;
- successful authentication and success after failures;
- sensitive account changes;
- protected authorisation denials;
- telemetry quality/availability problems (`100150`).

Starter alerts are pivots, not automatic proof of compromise.

Startup explicitly restarts the Wazuh manager after asserting the stack so an existing installation loads the current rule file after `git pull`.

Night Watch / Telemetry Health views

The current local provisioning script attempts to create:

- **NeoLabs — Operation Night Watch** — server-assigned pod baseline view with event type, identity, source, outcome, correlation/rule/original event-time fields;
- **NeoLabs — Telemetry Health** — rule `100150` / collection-parser-visibility troubleshooting view.

Saved-object provisioning is fail-soft. If a dashboard version/API rejects the import, Threat Hunting remains available and the Doctor reports the issue.

Freshness and local retention

- Default telemetry freshness warning: **90 minutes**.
- Default local `wazuh-alerts-*` retention: **30 days**.
- Disk warning: **85%**; critical: **92%**.

Retention applies only to the intern's local alert indices. It does not delete VCC telemetry archives, approved evidence or server-side records and does not force-overwrite unrelated existing ISM policies.

Workstation capacity

Run:

```
bash wazuh-stack/scripts/compatibility-check.sh
```

Current baseline:

Requirement	Hard floor	Preferred	Recommended
Linux/WSL2 visible memory	7 GiB	8 GiB	12-16 GiB
CPU	4 logical cores	4	6+
Free disk	25 GiB	25 GiB	50 GiB
Compose	v2	v2	current supported v2
<code>vm.max_map_count</code>	262144	262144	≥ 262144

Windows uses WSL2; Ubuntu specifically is **not** required. Kali, Debian and other current WSL2 distros are acceptable when Docker/Python/OpenSSL/curl are available. Git Bash/MSYS/Cygwin are not supported as the Wazuh runtime.

Manual/advanced stack operations

These remain useful for troubleshooting/advanced non-Windows operation:

```
bash scripts/compatibility-check.sh
bash scripts/generate-local-secrets.sh
bash scripts/prepare-stack.sh
bash scripts/preflight.sh
bash scripts/start.sh
bash scripts/health-check.sh
bash scripts/verify-telemetry-pipeline.sh --wait 180
bash scripts/telemetry-freshness.sh
bash scripts/doctor.sh
```

Use `scripts/backup.sh`, `verify-backup.sh`, `restore.sh`, `stop.sh` and `reset.sh` only according to their warnings. Destructive reset is local-only and must not be used as a substitute for diagnosing a normal Week 1 telemetry issue.

Private files

Never commit/share publicly:

- `.env` ;
- NeoLabs Access Codes/session files;
- VCC private keys/certificates/private signed URLs;
- local Wazuh passwords;
- enrolment/runtime state;
- raw evidence containing real/private information or another pod's data;
- generated backups.

Network/security design

- Dashboard binds to loopback by default.
- Indexer and Wazuh API are not published to the host for student use.
- Internal Wazuh components remain on the Compose internal network.
- Pod scope is server-managed, not selected through `POD_LABEL` /URL/query parameters.
- The collector/replay path rejects cross-pod/non-synthetic data.

Troubleshooting order

Prefer `CHECK-NEOLABS-SOC.cmd` / `.\neolabs.cmd doctor`. For manual review, check: NeoLabs auth/runtime → raw VCC event file → manager localfile/rules (`wazuh-logtest`) → Filebeat → indexer → dashboard/time filter → freshness/telemetry-health events.

Full decision tree: [../troubleshooting/WAZUH_SETUP_AND_TROUBLESHOOTING_GUIDE.md](#).